

Generics java

Juan Ignacio Raggio

May 3, 2025

Contents

1	Como funciona?	2
1.1	Permite que el compilador chequee si los tipos de datos son correspondidos	2
2	Como crear una clase generica?	2
2.1	Se pueden recibir la cantidad deseada de tipos en una clase generica	2
3	Como extender una clase generica?	2
4	Restriccion de tipo de dato generico	3
5	Interfaces for Generics	3
6	Wildcards	4
7	Parametrizacion de generics	4
8	Ojo porque los generics tienen que ser clases, no pueden ser tipos built-in	5
9	Como crear un array de tipo generico - Initializing a generic array	5

1 Como funciona?

1.1 Permite que el compilador chequee si los tipos de datos son correspondidos

- En tiempo de ejecucion es todo object y hay casteos

2 Como crear una clase generica?

```
public class Pair<T> {  
    private final T left, right;  
    public Pair(T left, T right) {  
        this.left = left;  
        this.right = right;  
    }  
    public T get_left() { return left; }  
    public T get_right() { return right; }  
}
```

2.1 Se pueden recibir la cantidad deseada de tipos en una clase generica

```
public class Xair <T, P, V, ...> {}
```

3 Como extender una clase generica?

- Cuando extendiendo una clase generica tengo que especificar el dato que va a usar esa extension
 - Ejemplo:

```
public abstract class PortManagement <T extends Port> { ... }
```

```
public class air_extraction_points extends PortManagement <Airport> {
```

Entonces para la clase `air_extraction_points`, la clase genrica `PortManagement`, va a usar el tipo `Airport`, es decir: `T` va a ser **reemplazado por `Airport`**, entonces luego dentro de `air_extraction_points` las variables que maneja la clase `PortManagement` van a tener acceso a los metodos de la clase `Airport`

4 Restriccion de tipo de dato generico

```
public class PortManagement <T extends Port> { ... }
```

Entonces T si o si va a tener que ser una extension de port o port

5 Interfaces for Generics

- Notemos en el siguiente ejemplo que todas las clases tienen que implementar `get` y `add` entonces nos conviene hacer una interfaz que sea intermediaria entre todas las tipos de listas

```
public class LinkedList<E> {  
    public E get(int index) { ... }  
    public void add(E elem) { ... }  
    ...  
}
```

```
public class ArrayList<E> {  
    public E get(int index) { ... }  
    public void add(E elem) { ... }  
    ...  
}
```

```
public class DoubleLinkedList<E> {  
    public E get(int index) { ... }  
    public void add(E elem) { ... }  
    ...  
}
```

- Mejor forma:

```
public interface List<E> {  
    boolean contains(E elem);  
    E get(int index);  
    void add(E elem);  
}  
public class LinkedList<E> {  
    public E get(int index) { ... }  
    public void add(E elem) { ... }
```

```

    ...
}

public class ArrayList<E> {
    public E get(int index) { ... }
    public void add(E elem) { ... }
    ...
}

public class DoubleLinkedList<E> {
    public E get(int index) { ... }
    public void add(E elem) { ... }
    ...
}

```

6 Wildcards

- En caso de usar Wildcards, es porque no nos interesa el tipo de dato generico

```

void show(List<?> aList) {
    ...
}

```

- Nos sirve para hacer metodos que sirvan para cualquier tipo de dato que sea hijo de algo que nosotros especificamos
- Ojo porque no podria retornar algo de tipo <?> pues no se que es

7 Parametrizacion de generics

```

public <Q extends Quotation> Q metodoRandom(Q var1, Q var2, Q var3);

```

8 Ojo porque los generics tienen que ser clases, no pueden ser tipos built-in

9 Como crear un array de tipo generico - Initializing a generic array

- Ejemplo seguro en un metodo next:

```
@SuppressWarnings("unchecked")
public T[] next() {
    T[] returnArray = (T[]) new Object[stepSize];
    for (int i = 0; i < returnArray.length; ++i) {
        returnArray[i] = collection[position + i];
    }
    ++position;
    return returnArray;
}
```

- Ejemplo de como NO hacerlo:
- Lo que tendria que haber hecho es como esta en el next, tener un array privado de tipo T y en el constructor hacer el casting
- En caso de que el usuario pase un arreglo de otro tipo va a fallar en tiempo de ejecucion

```
public class ArrayStack<E> implements Stack<E> {

    // Esta forma no es correcta, o no es la que espera la catedra
    private Object[] m_Elements;
    private int m_SavedElements;
    private static final int s_INITIALDIM = 10;

    private void resize() {
        m_Elements = Arrays.copyOf(m_Elements, m_SavedElements + s_INITIALDIM);
    }

    public ArrayStack() {
        m_Elements = new Object[s_INITIALDIM];
    }
}
```

```

// Pushes the element on top of the stack
public void push(E element) {
    if (m_SavedElements == m_Elements.length) {
        resize();
    }
    m_Elements[m_SavedElements++] = element;
}

// Returns the element at the top of the stack
@SuppressWarnings("unchecked")
public E peek() {
    if (isEmpty())
        throw new EmptyStackException();
    return (E) m_Elements[m_SavedElements - 1];
}

// Returns and removes the element at the top of the stack
public E pop() {
    E returnType = peek();
    --m_SavedElements;
    return returnType;
}

// True if empty stack
public boolean isEmpty() {
    return m_SavedElements == 0;
}

@SuppressWarnings("unchecked")
public String toString() {
    StringBuilder strBldr = new StringBuilder();
    strBldr.append("[");
    for (int i = m_SavedElements - 1; i > 0; --i) {
        strBldr.append((E) m_Elements[i]);
    }
    strBldr.append(", %s".formatted(((E) m_Elements[0]).toString()));
    return strBldr.toString();
}

```

}